TALLER DE PROGRAMACIÓN WEB II

Control de flujo interactivo: temporizadores y eventos personalizados

Universidad	Universidad de Caldas	**Asignatura**	Programación Web Creativa II
Docente	Andrés Felipe Osorio Quintero	**Perfil docente**	Ingeniero de Software, desarrollador web y marketing digital
Unidad	2. Interactividad avanzada y almacenamiento local		
Subtema	Control de flujo interactivo		
Documento fuente	1-Control_de_flujo_interactivo.pdf		
Código base	Codigo HTML CSS JS.docx (lista de tareas)		
Estudiante	Mariana Diaz Jaramillo, Samuel González Ramírez y Eliseth Nathaly Velasco Gómez	**Fecha**	29/10/2026

Indicaciones: responda las preguntas 1 a 5 con sus propias palabras. Desarrolle los ejercicios 6 a 20 en Visual Studio Code, tomando como punto de partida el código base y siguiendo el orden propuesto, y adjunte las evidencias solicitadas.

Parte I. Teoría Preguntas 1 a 5

1. Modelo de ejecución del navegador

Explique con sus propias palabras cómo ejecuta el navegador el código JavaScript de una página, relacionando el hilo principal, la pila de llamadas, las Web APIs, las colas de tareas y de microtareas, y el bucle de eventos. Con base en ese modelo, argumente por qué una promesa ya resuelta se atiende antes que un setTimeout con retardo cero.

Respuesta:

El navegador ejecuta el JavaScript en un único hilo principal. Cada función que se invoca entra en la pila de llamadas y se ejecuta hasta terminar. Cuando el código pide algo asíncrono (un temporizador, una petición de red, un evento del usuario), el trabajo se delega a las Web APIs del navegador, que lo atienden fuera de la pila. Al completarse, el callback no se ejecuta de inmediato: se encola. Los temporizadores y los eventos van a la cola de tareas (macrotareas); los then/catch de las promesas y queueMicrotask van a la cola de microtareas.

El bucle de eventos vigila constantemente la pila: cuando queda vacía, primero ejecuta todas las microtareas pendientes hasta agotarlas y solo después toma una macrotarea, deja que el navegador pinte si hace falta y repite el ciclo.

Por eso una promesa ya resuelta se atiende antes que un setTimeout con retardo cero: el retardo cero solo indica en qué momento el callback entra a la cola de macrotareas, pero esa cola no se consulta hasta haber vaciado las microtareas. El then de la promesa ya está en la cola de microtareas cuando termina el código síncrono, así que se ejecuta primero.

2. El retardo de un temporizador

¿Por qué se afirma que el retardo de setTimeout es un tiempo mínimo y no una garantía de puntualidad? Analice al menos dos situaciones que pueden retrasar un temporizador y explique qué consecuencias tienen para una cuenta regresiva que resta un segundo en cada ejecución.

Respuesta:
El retardo de setTimeout es un mínimo porque el temporizador solo garantiza que el callback no se ejecutará antes de ese tiempo. Al vencer, el callback se encola como macrotarea y debe esperar su turno hasta que la pila esté libre y se hayan procesado las microtareas y las tareas anteriores.

Situaciones que lo retrasan: (1) código síncrono pesado, como un bucle largo, que mantiene ocupado el hilo principal; (2) pestañas en segundo plano, donde el navegador limita los temporizadores (por ejemplo a uno por segundo) para ahorrar energía; también influyen la sobrecarga de CPU, las microtareas encadenadas y el anidamiento de temporizadores.

En una cuenta regresiva que resta un segundo en cada ejecución, cada ciclo tarda 1000 ms más un pequeño retraso, y esos retrasos se acumulan (deriva): el contador se atrasa respecto al reloj real y, con la pestaña en segundo plano, puede quedar muy desfasado. Lo correcto es guardar la hora de inicio con performance.now() o Date.now() y calcular el tiempo restante como diferencia con el reloj; el temporizador solo sirve para refrescar la pantalla.

3. setInterval frente a setTimeout recursivo

Compare setInterval y setTimeout recursivo. Argumente cuál utilizaría en un visor de cuenta regresiva y cuál en la consulta periódica a un servicio remoto, y explique qué precauciones debe tomar con los identificadores para evitar intervalos duplicados y fugas de memoria.

Respuesta:
setInterval repite el callback cada cierto intervalo fijo sin esperar a que la ejecución anterior termine; si la tarea es lenta o el hilo está ocupado, las ejecuciones pueden quedar amontonadas o muy seguidas. El setTimeout recursivo programa la siguiente ejecución al terminar la actual, lo que garantiza que no se solapen y permite cambiar la espera en cada vuelta (por ejemplo, un retroceso exponencial tras un error).

En un visor de cuenta regresiva usaría setInterval, porque solo se necesita un refresco regular y barato de la pantalla, mientras el tiempo real se calcula con el reloj. En la consulta periódica a un servicio remoto usaría setTimeout recursivo: así no se lanza una nueva consulta hasta recibir la respuesta anterior y se puede alargar la espera cuando el servicio falla.

Precauciones con los identificadores: guardar siempre el valor devuelto por setInterval/setTimeout en una variable accesible; antes de iniciar, comprobar si ya existe (id !== null) o limpiar el anterior para no duplicar intervalos; al pausar, terminar o destruir el componente llamar a clearInterval/clearTimeout y asignar null; en el caso recursivo, cancelar también el timeout pendiente y usar una bandera de estado. Así se evitan ejecuciones repetidas y fugas de memoria.

4. Eventos personalizados con CustomEvent

Explique para qué sirve CustomEvent y cómo influyen las opciones detail, bubbles, cancelable y composed en su comportamiento. Interprete por qué un evento emitido por un botón con los valores por defecto no llega a los escuchadores de document, y qué implica que dispatchEvent sea síncrono.

Respuesta:
CustomEvent permite crear eventos propios de la aplicación (por ejemplo tareas:agregada) con el mismo mecanismo de los eventos nativos: addEventListener y dispatchEvent. Así un módulo anuncia lo ocurrido sin conocer a quienes reaccionan, lo que reduce el acoplamiento.

detail transporta los datos del evento. bubbles indica si el evento sube por el árbol del DOM hacia los ancestros (por defecto es false). cancelable permite que un escuchador llame a preventDefault para impedir la acción asociada, y dispatchEvent devuelve false en ese caso. composed define si el evento puede cruzar la frontera de un Shadow DOM hacia el DOM exterior.

Con los valores por defecto (bubbles: false) el evento solo se dispara en el elemento emisor y no sube en la fase de burbujeo, por eso los escuchadores de document no lo reciben. Que dispatchEvent sea síncrono significa que todos los escuchadores se ejecutan completos, en orden de registro, antes de que dispatchEvent devuelva el control: el emisor puede leer de inmediato el resultado (true/false), pero un escuchador lento bloquea al emisor y uno registrado después del envío no verá ese evento.

5. Interfaz web y procesos ETL

Analice qué aportan los temporizadores y los eventos personalizados a un proceso de datos y qué no pueden hacer. Argumente por qué setInterval no es un planificador de procesos ETL y qué le corresponde al navegador, al servidor de aplicación y a la infraestructura ETL especializada.

Respuesta:
Los temporizadores y los eventos personalizados aportan, en la interfaz, la forma de organizar un proceso en el tiempo: refrescar el progreso, consultar el estado con reintentos, aplicar debounce, mostrar avisos y bitácoras, y desacoplar la pantalla del servicio que emite eventos como etl:progreso o etl:fin. También sirven para simular y visualizar un proceso.

No pueden ejecutar procesos de forma confiable: dependen de que la pestaña esté abierta, el navegador los frena en segundo plano, acumulan deriva, no sobreviven a una recarga y no ofrecen calendario, reintentos persistentes, transacciones ni auditoría. Por eso setInterval no es un planificador de procesos ETL: si el usuario cierra la pestaña el proceso muere y varios usuarios podrían lanzar ejecuciones duplicadas.

Reparto de responsabilidades: el navegador captura parámetros, solicita el inicio, muestra progreso y bitácora y permite cancelar; el servidor de aplicación autentica y autoriza, valida, expone la API (iniciar, consultar estado, cancelar) y registra las ejecuciones; la infraestructura ETL especializada (orquestador o planificador) programa y ejecuta extracción, transformación y carga, con reintentos, dependencias, monitoreo y escalabilidad.

Parte II. Práctica Ejercicios 6 a 20

6. Orden de ejecución

¿En qué orden se muestran en la consola un código síncrono, un setTimeout con retardo 0 y una promesa ya resuelta?

Actividad: Escriba un script con dos mensajes síncronos (inicio y fin), un setTimeout con retardo 0 y una promesa resuelta con then. Anote su predicción antes de ejecutarlo, verifíquela en la consola del navegador y compruebe que coincide.

Evidencia 1: captura del código en Visual Studio Code	Evidencia 2: captura del resultado en el navegador

ej06_orden.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 6 - Orden de ejecución</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Orden de ejecución</h1>
  <pre id="salida"></pre>
  <script>
    const salida = document.getElementById('salida');
    const log = m => {
      console.log(m);
      salida.textContent += m + '\n';
    };

    // PREDICCIÓN: inicio, fin, promesa, setTimeout
    log('inicio (síncrono)');
    setTimeout(() => log('setTimeout 0 (macrotarea)'), 0);
    Promise.resolve().then(() => log('promesa (microtarea)'));
    log('fin (síncrono)');
  </script>
</body>
</html>
```

Orden de ejecución

```
inicio (síncrono)
fin (síncrono)
promesa (microtarea)
setTimeout 0 (macrotarea)
```

Enlace de GitHub (opcional):

7. Mensaje temporal

¿Cómo confirmar al usuario que una tarea fue agregada sin que deba cerrar el mensaje manualmente?

Actividad: Sobre la lista de tareas del código base, muestre el texto “Tarea agregada” en un elemento con role="status" y ocúltelo a los 4 segundos con setTimeout, guardando el identificador del temporizador.

Evidencia 1: captura del código en Visual Studio Code	Evidencia 2: captura del resultado en el navegador

ej07_mensaje.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 7 - Mensaje temporal</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <p id="aviso" role="status" hidden></p>
  <ul id="lista"></ul>
  <script>
    const form = document.getElementById('form');
    const texto = document.getElementById('texto');
    const lista = document.getElementById('lista');
    const aviso = document.getElementById('aviso');
    let idAviso = null;

    form.addEventListener('submit', e => {
      e.preventDefault();
      const t = texto.value.trim();
      if (!t) return;
      const li = document.createElement('li');
      li.textContent = t;
      lista.append(li);
      texto.value = '';
      aviso.textContent = 'Tarea agregada';
      aviso.hidden = false;
      idAviso = setTimeout(() => {
        aviso.hidden = true;
      }, 4000);
    });
  </script>
</body>
</html>
```



Enlace de GitHub (opcional):

8. Reinicio del temporizador

¿Qué ocurre si se agregan dos tareas seguidas antes de que venzan los 4 segundos?

Actividad: Use clearTimeout para que cada nuevo mensaje reinicie la cuenta y el temporizador anterior no oculte prematuramente al nuevo. Centralice la programación en una función llamada programarCierre.

Evidencia 1: captura del código en Visual Studio Code

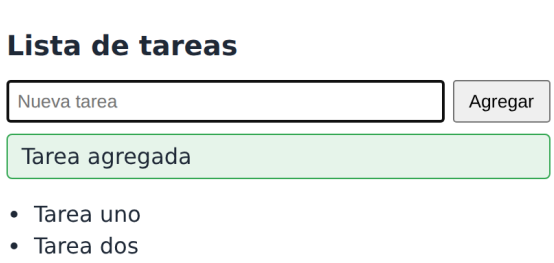
ej08_reinicio.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 8 - Reinicio del temporizador</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <p id="aviso" role="status" hidden></p>
  <ul id="lista"></ul>
  <script>
    const form = document.getElementById('form');
    const texto = document.getElementById('texto');
    const lista = document.getElementById('lista');
    const aviso = document.getElementById('aviso');
    let idAviso = null;

    function programarCierre() {
      clearTimeout(idAviso);   // anula el cierre anterior
      idAviso = setTimeout(() => {
        aviso.hidden = true;
      }, 4000);
    }

    form.addEventListener('submit', e => {
      e.preventDefault();
      const t = texto.value.trim();
      if (!t) return;
      const li = document.createElement('li');
      li.textContent = t;
      lista.append(li);
      texto.value = '';
      aviso.textContent = 'Tarea agregada';
      aviso.hidden = false;
      programarCierre();
    });
  </script>
</body>
</html>
```

Evidencia 2: captura del resultado en el navegador



Enlace de GitHub (opcional):

9. Pausa del aviso

¿Por qué un aviso que desaparece solo puede ser un problema de accesibilidad?

Actividad: Pause la cuenta del aviso mientras el puntero esté sobre él (mouseenter) y reinícielo al retirarlo (mouseleave).

Evidencia 1: captura del código en Visual Studio Code

• ej09_pausa.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 9 - Pausa del aviso</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <p id="aviso" role="status" hidden></p>
  <ul id="lista"></ul>
  <script>
    const form = document.getElementById('form');
    const texto = document.getElementById('texto');
    const lista = document.getElementById('lista');
    const aviso = document.getElementById('aviso');
    let idAviso = null;

    function programarCierre() {
      clearTimeout(idAviso);   // anula el cierre anterior
      idAviso = setTimeout(() => {
        aviso.hidden = true;
      }, 4000);
    }

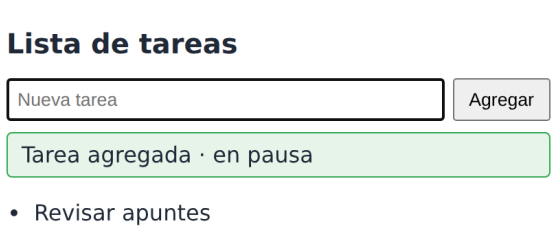
    form.addEventListener('submit', e => {
      e.preventDefault();
      const t = texto.value.trim();
      if (!t) return;
      const li = document.createElement('li');
      li.textContent = t;
      lista.append(li);
      texto.value = '';
      aviso.textContent = 'Tarea agregada';
      aviso.hidden = false;
      programarCierre();
    });

    // Pausa mientras el puntero está sobre el aviso
    aviso.addEventListener('mouseenter', () => {
      clearTimeout(idAviso);
      aviso.dataset.estado = 'pausa';
```

• ej09_pausa.html (continuación)

```html
    });
    aviso.addEventListener('mouseleave', () => {
      delete aviso.dataset.estado;
      programarCierre();
    });
  </script>
</body>
</html>
```

Evidencia 2: captura del resultado en el navegador



Enlace de GitHub (opcional):

10. Cuenta regresiva precisa

¿Cómo lograr que una cuenta regresiva no se atrase aunque el intervalo no sea puntual?

Actividad: Cree un visor de 10 segundos en formato mm:ss que se refresque cada 250 ms con setInterval y calcule el tiempo restante con performance.now(). Al llegar a cero, detenga el intervalo con clearInterval y muestre “¡Tiempo!”.

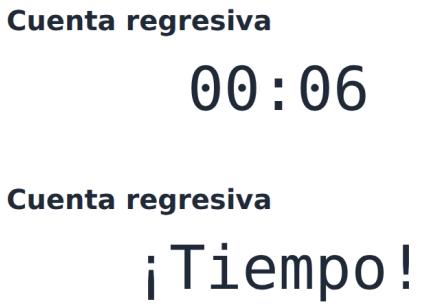
Evidencia 1: captura del código en Visual Studio Code	Evidencia 2: captura del resultado en el navegador

● ej10_cuenta.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 10 - Cuenta regresiva precisa</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Cuenta regresiva</h1>
  <p id="visor" class="visor">00:10</p>
  <script>
    const DURACION = 10000;   // 10 s en milisegundos
    const visor = document.getElementById('visor');

    const formato = ms => {
      const s = Math.ceil(ms / 1000);
      const mm = String(Math.floor(s / 60)).padStart(2, '0');
      const ss = String(s % 60).padStart(2, '0');
      return `${mm}:${ss}`;
    };

    const inicio = performance.now();
    const id = setInterval(() => {
      // se calcula con el reloj, no restando 250 ms
      const restante = DURACION - (performance.now() - inicio);
      if (restante <= 0) {
        clearInterval(id);
        visor.textContent = '¡Tiempo!';
        return;
      }
      visor.textContent = formato(restante);
    }, 250);
  </script>
</body>
</html>
```



Enlace de GitHub (opcional):

11. Controles de la cuenta regresiva

¿Qué sucede si el botón Iniciar se pulsa varias veces seguidas?

Actividad: Agregue los botones Iniciar, Pausar y Reiniciar al visor. Evite crear intervalos duplicados comprobando el identificador guardado, y conserve el tiempo restante al pausar para poder reanudar.

Evidencia 1: captura del código en Visual Studio Code	Evidencia 2: captura del resultado en el navegador

• ej11_controles.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 11 - Controles de la cuenta regresiva</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Cuenta regresiva</h1>
  <p id="visor" class="visor">00:10</p>
  <div class="fila">
    <button id="iniciar">Iniciar</button>
    <button id="pausar">Pausar</button>
    <button id="reiniciar">Reiniciar</button>
  </div>
  <script>
    const DURACION = 10000;
    const visor = document.getElementById('visor');
    let restante = DURACION;   // ms que faltan
    let id = null;             // id del intervalo
    let t0 = 0;                // marca del último inicio

    const formato = ms => {
      const s = Math.ceil(ms / 1000);
      const mm = String(Math.floor(s / 60)).padStart(2, '0');
      const ss = String(s % 60).padStart(2, '0');
      return `${mm}:${ss}`;
    };

    function tick() {
      const r = restante - (performance.now() - t0);
      if (r <= 0) {
        clearInterval(id);
        id = null;
        restante = 0;
        visor.textContent = '¡Tiempo!';
        return;
      }
      visor.textContent = formato(r);
    }

    iniciar.onclick = () => {
      if (id !== null || restante <= 0) return;  // sin duplicados
      t0 = performance.now();
      id = setInterval(tick, 250);
    };
```

• ej11_controles.html (continuación)

```
    pausar.onclick = () => {
      if (id === null) return;
      restante -= performance.now() - t0;        // conserva el tiempo
      clearInterval(id);
      id = null;
    };
    reiniciar.onclick = () => {
      clearInterval(id);
      id = null;
      restante = DURACION;
      visor.textContent = formato(restante);
    };
  </script>
</body>
</html>
```



Enlace de GitHub (opcional):

12. Consulta periódica con reintentos

¿Por qué es preferible un setTimeout recursivo a un setInterval para consultar un servicio?

Actividad: Simule una función consultarEstado con una promesa y un setTimeout de 300 ms que falle de forma aleatoria. Consulte el estado cada segundo; ante un error, duplique la espera hasta un máximo de 8 s. Registre cada resultado en una bitácora y detenga el ciclo al llegar a 100 % o con un botón Detener.

Evidencia 1: captura del código en Visual Studio Code

ej12_consulta.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 12 - Consulta periódica con reintentos</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Consulta de estado</h1>
  <div class="fila">
    <button id="iniciar">Iniciar</button>
    <button id="detener">Detener</button>
  </div>
  <pre id="bitacora"></pre>
  <script>
    const bitacora = document.getElementById('bitacora');
    const registrar = m => {
      const hora = new Date().toLocaleTimeString();
      bitacora.textContent += `[${hora}] ${m}\n`;
    };
    let progreso = 0, espera = 1000;
    let activo = false, idCiclo = null;

    function consultarEstado() {
      return new Promise((ok, fallo) => {
        setTimeout(() => {
          if (Math.random() < 0.4) {
            return fallo(new Error('Servicio no disponible'));
          }
          progreso = Math.min(100,
            progreso + 10 + Math.floor(Math.random() * 20));
          ok(progreso);
        }, 300);
      });
    }

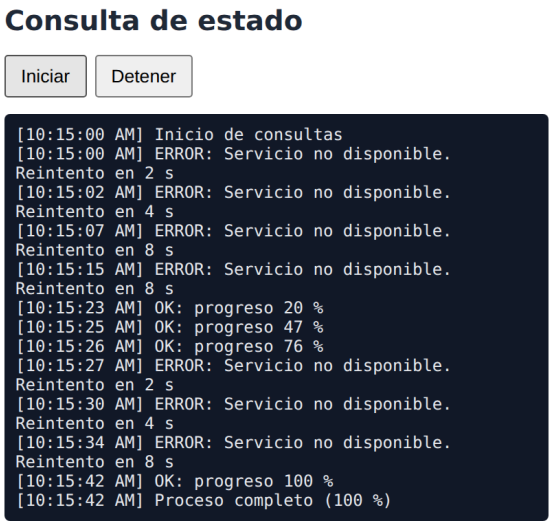
    async function ciclo() {
      if (!activo) return;
      try {
        const p = await consultarEstado();
        registrar(`OK: progreso ${p} %`);
        espera = 1000;
        if (p >= 100) return detener('Proceso completo (100
%)');
      } catch (err) {
        espera = Math.min(espera * 2, 8000);   // backoff
        registrar(`ERROR: ${err.message}. ` +
```

ej12_consulta.html (continuación)

```
          `Reintento en ${espera / 1000} s`);
      }
      if (activo) idCiclo = setTimeout(ciclo, espera);
    }

    function detener(motivo = 'Detenido por el usuario') {
      activo = false;
      clearTimeout(idCiclo);
      registrar(motivo);
    }
    iniciar.onclick = () => {
      if (activo) return;
      activo = true;
      progreso = 0;
      espera = 1000;
      registrar('Inicio de consultas');
      ciclo();
    };
    document.getElementById('detener').onclick = () =>
detener();
  </script>
</body>
</html>
```

Evidencia 2: captura del resultado en el navegador



Enlace de GitHub (opcional):

13. Filtro con debounce

¿Cómo evitar que un filtro se ejecute en cada pulsación de tecla?

Actividad: Agregue un campo de búsqueda que filtre las tareas de la lista aplicando debounce de 300 ms con un único setTimeout. Muestre el número de resultados en un elemento con role="status".

Evidencia 1: captura del código en Visual Studio Code

ej13_debounce.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 13 - Filtro con debounce</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <input id="busqueda" type="search" placeholder="Buscar...">
  <p id="resultados" role="status"></p>
  <ul id="lista"></ul>
  <script>
    const ini = ['Comprar leche', 'Estudiar JavaScript',
      'Enviar informe', 'Llamar al docente',
      'Estudiar para el parcial'];
    const form = document.getElementById('form');
    const texto = document.getElementById('texto');
    const lista = document.getElementById('lista');
    const busqueda = document.getElementById('busqueda');
    const resultados = document.getElementById('resultados');
    let idDebounce = null;

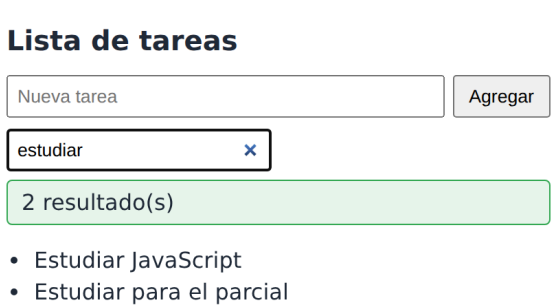
    function agregar(t) {
      const li = document.createElement('li');
      li.textContent = t;
      lista.append(li);
    }
    function filtrar() {
      const q = busqueda.value.trim().toLowerCase();
      let n = 0;
      for (const li of lista.children) {
        const ok = li.textContent.toLowerCase().includes(q);
        li.hidden = !ok;
        if (ok) n++;
      }
      resultados.textContent = `${n} resultado(s)`;
    }

    form.addEventListener('submit', e => {
      e.preventDefault();
      if (texto.value.trim()) agregar(texto.value.trim());
```

ej13_debounce.html (continuación)

```html
      texto.value = '';
      filtrar();
    });
    busqueda.addEventListener('input', () => {
      clearTimeout(idDebounce);          // un solo temporizador
      idDebounce = setTimeout(filtrar, 300);
    });
    ini.forEach(agregar);
    filtrar();
  </script>
</body>
</html>
```

Evidencia 2: captura del resultado en el navegador



Enlace de GitHub (opcional):

14. Primer evento personalizado

¿Cómo anunciar un hecho propio de la aplicación con el mismo mecanismo de los eventos nativos?

Actividad: Al agregar una tarea, emita un CustomEvent llamado tareas:agregada con bubbles: true y un detail con el texto de la tarea. Escúchelo en document y muestre el detail en la consola.

Evidencia 1: captura del código en Visual Studio Code

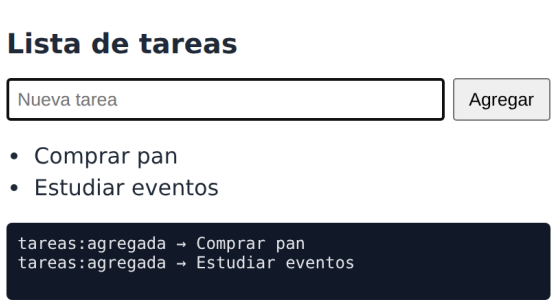
ej14_evento.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 14 - Primer evento personalizado</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <ul id="lista"></ul>
  <pre id="consola"></pre>
  <script>
    const consola = document.getElementById('consola');
    const log = m => {
      console.log(m);
      consola.textContent += m + '\n';
    };
    const form = document.getElementById('form');
    const texto = document.getElementById('texto');
    const lista = document.getElementById('lista');

    form.addEventListener('submit', e => {
      e.preventDefault();
      const t = texto.value.trim();
      if (!t) return;
      const li = document.createElement('li');
      li.textContent = t;
      lista.append(li);
      texto.value = '';
      li.dispatchEvent(new CustomEvent('tareas:agregada', {
        bubbles: true,            // sube hasta document
        detail: { texto: t }
      }));
    });

    document.addEventListener('tareas:agregada', e => {
      log('tareas:agregada → ' + e.detail.texto);
    });
  </script>
</body>
</html>
```

Evidencia 2: captura del resultado en el navegador



Enlace de GitHub (opcional):

15. Módulos desacoplados

¿Cómo actualizar un contador sin que el código de la lista llame funciones del contador?

Actividad: Cree un contador de tareas pendientes que solo escuche los eventos tareas:agregada y tareas:eliminada, y actualice su total con textContent. Emita ambos eventos desde el código de la lista.

Evidencia 1: captura del código en Visual Studio Code

ej15_contador.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 15 - Módulos desacoplados</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <p>Pendientes: <strong id="contador">0</strong></p>
  <ul id="lista"></ul>
  <script>
    const form = document.getElementById('form');
    const texto = document.getElementById('texto');
    const lista = document.getElementById('lista');

    function emitir(nombre, detail) {
      lista.dispatchEvent(new CustomEvent(nombre, {
        bubbles: true, detail
      }));
    }

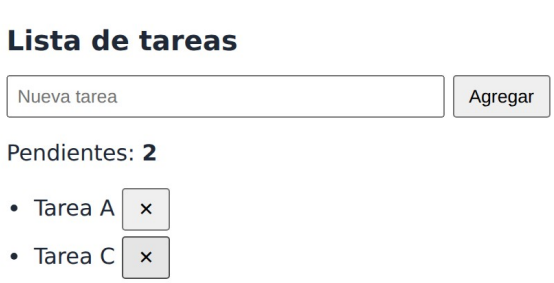
    form.addEventListener('submit', e => {
      e.preventDefault();
      const t = texto.value.trim();
      if (!t) return;
      const li = document.createElement('li');
      li.textContent = t + ' ';
      const b = document.createElement('button');
      b.textContent = '×';
      b.onclick = () => {
        li.remove();
        emitir('tareas:eliminada', { texto: t });
      };
      li.append(b);
      lista.append(li);
      texto.value = '';
      emitir('tareas:agregada', { texto: t });
    });

    // ---- Módulo contador: no conoce a la lista ----
    const total = document.getElementById('contador');
```

ej15_contador.html (continuación)

```html
    document.addEventListener('tareas:agregada', () => {
      total.textContent = Number(total.textContent) + 1;
    });
    document.addEventListener('tareas:eliminada', () => {
      total.textContent = Number(total.textContent) - 1;
    });
  </script>
</body>
</html>
```

Evidencia 2: captura del resultado en el navegador



Enlace de GitHub (opcional):

16. Evento cancelable

¿Cómo permitir que otro módulo impida una acción antes de que ocurra?

Actividad: Antes de agregar una tarea, emita tareas:antes-de-agregar con cancelable: true. Un validador independiente debe llamar a preventDefault si el texto tiene menos de 3 caracteres e informar el motivo; la lista solo agrega la tarea si dispatchEvent devuelve true.

Evidencia 1: captura del código en Visual Studio Code

ej16_cancelable.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 16 - Evento cancelable</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <p id="motivo" role="alert" hidden></p>
  <ul id="lista"></ul>
  <script>
    const form = document.getElementById('form');
    const texto = document.getElementById('texto');
    const lista = document.getElementById('lista');

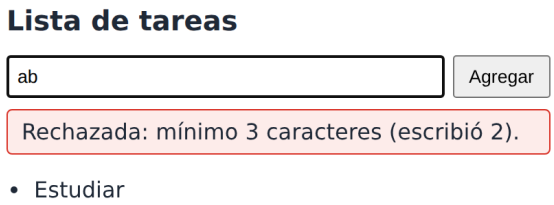
    // ---- Módulo lista ----
    form.addEventListener('submit', e => {
      e.preventDefault();
      const t = texto.value.trim();
      const antes = new CustomEvent('tareas:antes-de-agregar', {
        cancelable: true,
        detail: { texto: t }
      });
      // dispatchEvent devuelve false si alguien canceló
      if (!document.dispatchEvent(antes)) return;
      const li = document.createElement('li');
      li.textContent = t;
      lista.append(li);
      texto.value = '';
    });

    // ---- Módulo validador (independiente) ----
    const motivo = document.getElementById('motivo');
    document.addEventListener('tareas:antes-de-agregar', e => {
      const ok = e.detail.texto.length >= 3;
      motivo.hidden = ok;
      if (!ok) {
        e.preventDefault();
        motivo.textContent =
          'Rechazada: mínimo 3 caracteres (escribió ' +
          e.detail.texto.length + ').';
```

ej16_cancelable.html (continuación)

```
      }
    });
  </script>
</body>
</html>
```

Evidencia 2: captura del resultado en el navegador



Enlace de GitHub (opcional):

17. Retiro de escuchadores

¿Cómo evitar acciones repetidas y fugas de memoria al dejar de usar un componente?

Actividad: Registre los escuchadores del contador con la opción { signal } de un AbortController y agregue un botón “Desactivar contador” que llame a abort(). Registre además un escuchador con { once: true } y compruebe en la consola que ambos dejan de responder como corresponde.

Evidencia 1: captura del código en Visual Studio Code

ej17_retiro.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 17 - Retiro de escuchadores</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <p>Pendientes: <strong id="contador">0</strong></p>
  <button id="desactivar">Desactivar contador</button>
  <ul id="lista"></ul>
  <pre id="consola"></pre>
  <script>
    const consola = document.getElementById('consola');
    const log = m => {
      console.log(m);
      consola.textContent += m + '\n';
    };
    const form = document.getElementById('form');
    const texto = document.getElementById('texto');
    const lista = document.getElementById('lista');
    function emitir(nombre, detail) {
      lista.dispatchEvent(new CustomEvent(nombre, {
        bubbles: true, detail
      }));
    }
    form.addEventListener('submit', e => {
      e.preventDefault();
      const t = texto.value.trim();
      if (!t) return;
      const li = document.createElement('li');
      li.textContent = t;
      lista.append(li);
      texto.value = '';
      emitir('tareas:agregada', { texto: t });
    });

    // ---- Contador con AbortController ----
    const total = document.getElementById('contador');
    const ctl = new AbortController();
    const { signal } = ctl;
```

ej17_retiro.html (continuación)

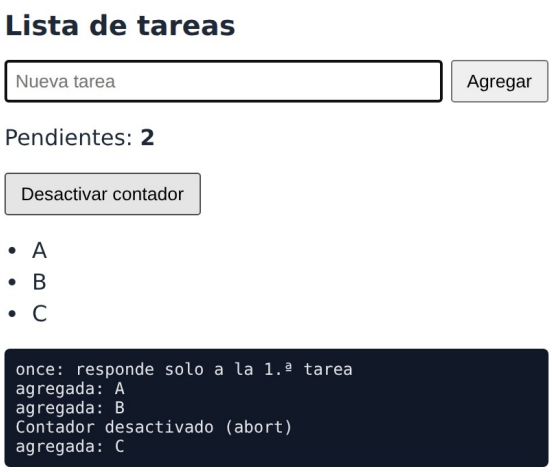
```html
    document.addEventListener('tareas:agregada', () => {
      total.textContent = Number(total.textContent) + 1;
    }, { signal });

    // Escuchador de una sola vez
    document.addEventListener('tareas:agregada', () => {
      log('once: responde solo a la 1.ª tarea');
    }, { once: true });

    // Escuchador permanente de control
    document.addEventListener('tareas:agregada', e => {
      log('agregada: ' + e.detail.texto);
    });

    desactivar.onclick = () => {
      ctl.abort();   // retira todo lo registrado con signal
      log('Contador desactivado (abort)');
    };
  </script>
</body>
</html>
```

Evidencia 2: captura del resultado en el navegador



Enlace de GitHub (opcional):

18. Servicio con EventTarget

¿Cómo aplicar el modelo de eventos a la lógica de negocio sin depender del DOM?

Actividad: Cree una clase GestorTareas que extienda EventTarget, con los métodos agregar y eliminar que emitan eventos personalizados. La interfaz debe escuchar esos eventos para dibujar la lista.

Evidencia 1: captura del código en Visual Studio Code

ej18_gestor.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 18 - Servicio con EventTarget</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <ul id="lista"></ul>
  <script>
    class GestorTareas extends EventTarget {
      #tareas  = [];
      #sig  = 0;
      agregar(texto) {
        const t = { id: ++this.#sig , texto };
        this.#tareas .push(t);
        this.dispatchEvent(
          new CustomEvent('tareas:agregada', { detail: t }));
        return t;
      }
      eliminar(id) {
        const i = this.#tareas .findIndex(t => t.id === id);
        if (i < 0) return;
        const [t] = this.#tareas .splice(i, 1);
        this.dispatchEvent(
          new CustomEvent('tareas:eliminada', { detail: t }));
      }
    }

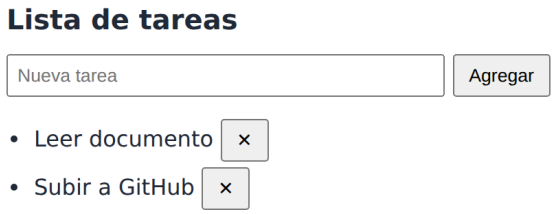
    // ---- Interfaz: solo escucha y dibuja ----
    const gestor = new GestorTareas();
    const lista = document.getElementById('lista');
    const texto = document.getElementById('texto');

    gestor.addEventListener('tareas:agregada', e => {
      const li = document.createElement('li');
      li.dataset.id = e.detail.id;
      li.textContent = e.detail.texto + ' ';
      const b = document.createElement('button');
      b.textContent = '×';
      b.onclick = () => gestor.eliminar(e.detail.id);
```

ej18_gestor.html (continuación)

```
      li.append(b);
      lista.append(li);
    });
    gestor.addEventListener('tareas:eliminada', e => {
      lista.querySelector(`[data-id="${e.detail.id}"]`)
        ?.remove();
    });
    document.getElementById('form').onsubmit = e => {
      e.preventDefault();
      if (texto.value.trim())
  gestor.agregar(texto.value.trim());
      texto.value = '';
    };
  </script>
</body>
</html>
```

Evidencia 2: captura del resultado en el navegador



Enlace de GitHub (opcional):

19. Temporizador y evento combinados

¿Cómo avisar que un tiempo venció sin que el temporizador conozca a quién debe reaccionar?

Actividad: Permita asignar a cada tarea un tiempo límite en segundos con setTimeout. Al vencer, emita tareas:vencida con un detail con los datos de la tarea; otro componente la escucha y muestra el aviso. Cancele el temporizador con clearTimeout si la tarea se elimina antes.

Evidencia 1: captura del código en Visual Studio Code

ej19_vencida.html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 19 - Temporizador y evento combinados</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Tareas con tiempo límite</h1>
  <form id="form">
    <input id="texto" placeholder="Tarea">
    <input id="limite" type="number" min="1" value="3"
      style="max-width:70px" title="segundos">
    <button>Agregar</button>
  </form>
  <p id="aviso" role="alert" hidden></p>
  <ul id="lista"></ul>
  <script>
    const lista = document.getElementById('lista');
    const texto = document.getElementById('texto');
    const limite = document.getElementById('limite');
    const temporizadores = new Map();   // id → id del setTimeout
    let sig = 0;

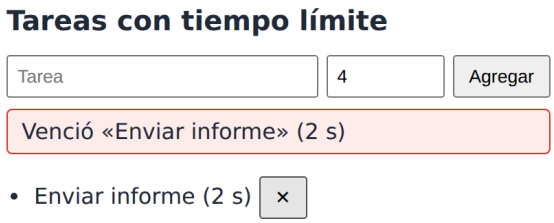
    function agregar(t, seg) {
      const id = ++sig;
      const li = document.createElement('li');
      li.dataset.id = id;
      li.textContent = `${t} (${seg} s) `;
      const b = document.createElement('button');
      b.textContent = '×';
      b.onclick = () => eliminar(id, li);
      li.append(b);
      lista.append(li);
      temporizadores.set(id, setTimeout(() => {
        temporizadores.delete(id);
        li.dispatchEvent(new CustomEvent('tareas:vencida', {
          bubbles: true,
          detail: { id, texto: t, limite: seg }
        }));
      }, seg * 1000));
    }
    function eliminar(id, li) {
      clearTimeout(temporizadores.get(id));   // evita avisos fantasma
      temporizadores.delete(id);
      li.remove();
```

ej19_vencida.html (continuación)

```html
    }
    document.getElementById('form').onsubmit = e => {
      e.preventDefault();
      const t = texto.value.trim();
      if (t) agregar(t, Number(limite.value) || 1);
      texto.value = '';
    };

    // ---- Componente de avisos (solo escucha) ----
    const aviso = document.getElementById('aviso');
    document.addEventListener('tareas:vencida', e => {
      aviso.textContent =
        `Venció «${e.detail.texto}» (${e.detail.limite} s)`;
      aviso.hidden = false;
    });
  </script>
</body>
</html>
```

Evidencia 2: captura del resultado en el navegador



Enlace de GitHub (opcional):

20. Caso integrador: monitor ETL simulado

¿Cómo separar un servicio que conduce un proceso en el tiempo de la interfaz que lo muestra?

Actividad: Cree una clase que extienda EventTarget y simule un proceso de tres etapas (Extracción, Transformación y Carga) con setInterval y performance.now(), emitiendo eventos con prefijo etl:. La interfaz debe mostrar una barra de progreso, una bitácora y un botón Cancelar, y rechazar el inicio mediante un evento cancelable si no se eligió un origen de datos.

Evidencia 1: captura del código en Visual Studio Code

ej20_etl.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 20 - Monitor ETL simulado</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Monitor ETL</h1>
  <div class="fila">
    <select id="origen">
      <option value="">— elija un origen —</option>
      <option>ventas.csv</option>
      <option>clientes.json</option>
    </select>
    <button id="iniciar">Iniciar</button>
    <button id="cancelar">Cancelar</button>
  </div>
  <progress id="barra" max="100" value="0"></progress>
  <pre id="bitacora"></pre>
  <script>
    class MonitorETL extends EventTarget {
      #id   = null;
      #etapas  = ['Extracción', 'Transformación', 'Carga'];
      #total   = 6000;   // duración simulada (ms)
      #emitir (n, detail = {}) {
        this.dispatchEvent(new CustomEvent('etl:' + n, { detail
}));
      }
      iniciar(origen) {
        if (this.#id   !== null) return false;
        const antes = new CustomEvent('etl:antes-de-iniciar', {
          cancelable: true, detail: { origen }
        });
        if (!this.dispatchEvent(antes)) {
          this.#emitir ('rechazado', { motivo: 'sin origen' });
          return false;
        }
        const t0 = performance.now();
        let previa = -1;
        this.#emitir ('inicio', { origen });
        this.#id   = setInterval(() => {
          const p = Math.min(1, (performance.now() - t0) /
this.#total );
          const e = Math.min(2, Math.floor(p * 3));
          if (e !== previa) {
            previa = e;
            this.#emitir ('etapa', { nombre: this.#etapas [e]
});
```

ej20_etl.html (continuación)

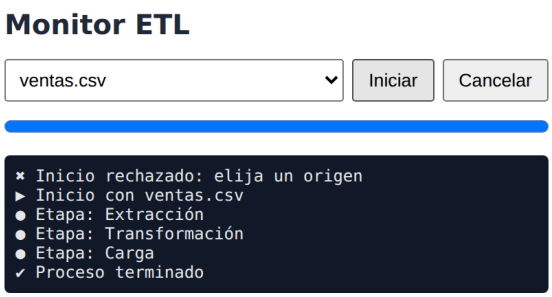
```
          }
          this.#emitir ('progreso', { pct: Math.round(p * 100)
});
          if (p >= 1) {
            clearInterval(this.#id );
            this.#id   = null;
            this.#emitir ('fin');
          }
        }, 200);
        return true;
      }
      cancelar() {
        if (this.#id   === null) return;
        clearInterval(this.#id );
        this.#id   = null;
        this.#emitir ('cancelado');
      }
    }

    // ---- Interfaz ----
    const etl = new MonitorETL();
    const barra = document.getElementById('barra');
    const bitacora = document.getElementById('bitacora');
    const anotar = m => bitacora.textContent += m + '\n';

    // Validador independiente: exige un origen
    etl.addEventListener('etl:antes-de-iniciar', e => {
      if (!e.detail.origen) e.preventDefault();
    });
    etl.addEventListener('etl:rechazado', () =>
      anotar('✖ Inicio rechazado: elija un origen'));
    etl.addEventListener('etl:inicio', e =>
      anotar('▶ Inicio con ' + e.detail.origen));
    etl.addEventListener('etl:etapa', e =>
      anotar('● Etapa: ' + e.detail.nombre));
    etl.addEventListener('etl:progreso', e =>
      barra.value = e.detail.pct);
    etl.addEventListener('etl:fin', () => anotar('✔ Proceso
terminado'));
    etl.addEventListener('etl:cancelado', () => anotar('■
Cancelado'));

    iniciar.onclick = () => etl.iniciar(origen.value);
    cancelar.onclick = () => etl.cancelar();
  </script>
</body>
</html>
```

Evidencia 2: captura del resultado en el navegador



Enlace de GitHub (opcional):

Entrega final

Pegue el enlace completo del repositorio de GitHub donde se encuentran todos los ejercicios.

Enlace del repositorio	

Anexo. Código fuente completo de los ejercicios 6 a 20

Cada ejercicio es un archivo HTML independiente que usa la hoja de estilos estilos.css (incluida al final del anexo).

Ejercicio 6: ej06_orden.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 6 - Orden de ejecución</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Orden de ejecución</h1>
  <pre id="salida"></pre>
  <script>
    const salida = document.getElementById('salida');
    const log = m => {
      console.log(m);
      salida.textContent += m + '\n';
    };

    // PREDICCIÓN: inicio, fin, promesa, setTimeout
    log('inicio (síncrono)');
    setTimeout(() => log('setTimeout 0 (macrotarea)'), 0);
    Promise.resolve().then(() => log('promesa (microtarea)'));
    log('fin (síncrono)');
  </script>
</body>
</html>
```

Ejercicio 7: ej07_mensaje.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 7 - Mensaje temporal</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <p id="aviso" role="status" hidden></p>
  <ul id="lista"></ul>
  <script>
    const form = document.getElementById('form');
    const texto = document.getElementById('texto');
    const lista = document.getElementById('lista');
    const aviso = document.getElementById('aviso');
    let idAviso = null;

    form.addEventListener('submit', e => {
      e.preventDefault();
      const t = texto.value.trim();
      if (!t) return;
      const li = document.createElement('li');
      li.textContent = t;
      lista.append(li);
      texto.value = '';
      aviso.textContent = 'Tarea agregada';
      aviso.hidden = false;
      idAviso = setTimeout(() => {
        aviso.hidden = true;
      }, 4000);
    });
  </script>
</body>
</html>
```

Ejercicio 8: ej08_reinicio.html

```
<!DOCTYPE html>
```

```
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 8 - Reinicio del temporizador</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <p id="aviso" role="status" hidden></p>
  <ul id="lista"></ul>
  <script>
    const form = document.getElementById('form');
    const texto = document.getElementById('texto');
    const lista = document.getElementById('lista');
    const aviso = document.getElementById('aviso');
    let idAviso = null;

    function programarCierre() {
      clearTimeout(idAviso);   // anula el cierre anterior
      idAviso = setTimeout(() => {
        aviso.hidden = true;
      }, 4000);
    }

    form.addEventListener('submit', e => {
      e.preventDefault();
      const t = texto.value.trim();
      if (!t) return;
      const li = document.createElement('li');
      li.textContent = t;
      lista.append(li);
      texto.value = '';
      aviso.textContent = 'Tarea agregada';
      aviso.hidden = false;
      programarCierre();
    });
  </script>
</body>
</html>
```

Ejercicio 9: ej09_pausa.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 9 - Pausa del aviso</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <p id="aviso" role="status" hidden></p>
  <ul id="lista"></ul>
  <script>
    const form = document.getElementById('form');
    const texto = document.getElementById('texto');
    const lista = document.getElementById('lista');
    const aviso = document.getElementById('aviso');
    let idAviso = null;

    function programarCierre() {
      clearTimeout(idAviso);   // anula el cierre anterior
      idAviso = setTimeout(() => {
        aviso.hidden = true;
      }, 4000);
    }

    form.addEventListener('submit', e => {
      e.preventDefault();
```

```
      const t = texto.value.trim();
      if (!t) return;
      const li = document.createElement('li');
      li.textContent = t;
      lista.append(li);
      texto.value = '';
      aviso.textContent = 'Tarea agregada';
      aviso.hidden = false;
      programarCierre();
    });

    // Pausa mientras el puntero está sobre el aviso
    aviso.addEventListener('mouseenter', () => {
      clearTimeout(idAviso);
      aviso.dataset.estado = 'pausa';
    });
    aviso.addEventListener('mouseleave', () => {
      delete aviso.dataset.estado;
      programarCierre();
    });
  </script>
</body>
</html>
```

Ejercicio 10: ej10_cuenta.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 10 - Cuenta regresiva precisa</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Cuenta regresiva</h1>
  <p id="visor" class="visor">00:10</p>
  <script>
    const DURACION = 10000;   // 10 s en milisegundos
    const visor = document.getElementById('visor');

    const formato = ms => {
      const s = Math.ceil(ms / 1000);
      const mm = String(Math.floor(s / 60)).padStart(2, '0');
      const ss = String(s % 60).padStart(2, '0');
      return `${mm}:${ss}`;
    };

    const inicio = performance.now();
    const id = setInterval(() => {
      // se calcula con el reloj, no restando 250 ms
      const restante = DURACION - (performance.now() - inicio);
      if (restante <= 0) {
        clearInterval(id);
        visor.textContent = '¡Tiempo!';
        return;
      }
      visor.textContent = formato(restante);
    }, 250);
  </script>
</body>
</html>
```

Ejercicio 11: ej11_controles.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 11 - Controles de la cuenta regresiva</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Cuenta regresiva</h1>
  <p id="visor" class="visor">00:10</p>
  <div class="fila">
    <button id="iniciar">Iniciar</button>
    <button id="pausar">Pausar</button>
```

```
    <button id="reiniciar">Reiniciar</button>
  </div>
  <script>
    const DURACION = 10000;
    const visor = document.getElementById('visor');
    let restante = DURACION;   // ms que faltan
    let id = null;             // id del intervalo
    let t0 = 0;                // marca del último inicio

    const formato = ms => {
      const s = Math.ceil(ms / 1000);
      const mm = String(Math.floor(s / 60)).padStart(2, '0');
      const ss = String(s % 60).padStart(2, '0');
      return `${mm}:${ss}`;
    };

    function tick() {
      const r = restante - (performance.now() - t0);
      if (r <= 0) {
        clearInterval(id);
        id = null;
        restante = 0;
        visor.textContent = '¡Tiempo!';
        return;
      }
      visor.textContent = formato(r);
    }

    iniciar.onclick = () => {
      if (id !== null || restante <= 0) return;  // sin duplicados
      t0 = performance.now();
      id = setInterval(tick, 250);
    };
    pausar.onclick = () => {
      if (id === null) return;
      restante -= performance.now() - t0;        // conserva el tiempo
      clearInterval(id);
      id = null;
    };
    reiniciar.onclick = () => {
      clearInterval(id);
      id = null;
      restante = DURACION;
      visor.textContent = formato(restante);
    };
  </script>
</body>
</html>
```

Ejercicio 12: ej12_consulta.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 12 - Consulta periódica con reintentos</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Consulta de estado</h1>
  <div class="fila">
    <button id="iniciar">Iniciar</button>
    <button id="detener">Detener</button>
  </div>
  <pre id="bitacora"></pre>
  <script>
    const bitacora = document.getElementById('bitacora');
    const registrar = m => {
      const hora = new Date().toLocaleTimeString();
      bitacora.textContent += `[${hora}] ${m}\n`;
    };
    let progreso = 0, espera = 1000;
    let activo = false, idCiclo = null;

    function consultarEstado() {
      return new Promise((ok, fallo) => {
        setTimeout(() => {
```

```
          if (Math.random() < 0.4) {
            return fallo(new Error('Servicio no disponible'));
          }
          progreso = Math.min(100,
            progreso + 10 + Math.floor(Math.random() * 20));
          ok(progreso);
        }, 300);
      });
    }

    async function ciclo() {
      if (!activo) return;
      try {
        const p = await consultarEstado();
        registrar(`OK: progreso ${p} %`);
        espera = 1000;
        if (p >= 100) return detener('Proceso completo (100 %)');
      } catch (err) {
        espera = Math.min(espera * 2, 8000);   // backoff
        registrar(`ERROR: ${err.message}. ` +
          `Reintento en ${espera / 1000} s`);
      }
      if (activo) idCiclo = setTimeout(ciclo, espera);
    }

    function detener(motivo = 'Detenido por el usuario') {
      activo = false;
      clearTimeout(idCiclo);
      registrar(motivo);
    }
    iniciar.onclick = () => {
      if (activo) return;
      activo = true;
      progreso = 0;
      espera = 1000;
      registrar('Inicio de consultas');
      ciclo();
    };
    document.getElementById('detener').onclick = () => detener();
  </script>
</body>
</html>
```

Ejercicio 13: ej13_debounce.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 13 - Filtro con debounce</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <input id="busqueda" type="search" placeholder="Buscar...">
  <p id="resultados" role="status"></p>
  <ul id="lista"></ul>
  <script>
    const ini = ['Comprar leche', 'Estudiar JavaScript',
      'Enviar informe', 'Llamar al docente',
      'Estudiar para el parcial'];
    const form = document.getElementById('form');
    const texto = document.getElementById('texto');
    const lista = document.getElementById('lista');
    const busqueda = document.getElementById('busqueda');
    const resultados = document.getElementById('resultados');
    let idDebounce = null;

    function agregar(t) {
      const li = document.createElement('li');
      li.textContent = t;
      lista.append(li);
    }
```

```
    function filtrar() {
      const q = busqueda.value.trim().toLowerCase();
      let n = 0;
      for (const li of lista.children) {
        const ok = li.textContent.toLowerCase().includes(q);
        li.hidden = !ok;
        if (ok) n++;
      }
      resultados.textContent = `${n} resultado(s)`;
    }

    form.addEventListener('submit', e => {
      e.preventDefault();
      if (texto.value.trim()) agregar(texto.value.trim());
      texto.value = '';
      filtrar();
    });
    busqueda.addEventListener('input', () => {
      clearTimeout(idDebounce);          // un solo temporizador
      idDebounce = setTimeout(filtrar, 300);
    });
    ini.forEach(agregar);
    filtrar();
  </script>
</body>
</html>
```

Ejercicio 14: ej14_evento.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 14 - Primer evento personalizado</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <ul id="lista"></ul>
  <pre id="consola"></pre>
  <script>
    const consola = document.getElementById('consola');
    const log = m => {
      console.log(m);
      consola.textContent += m + '\n';
    };
    const form = document.getElementById('form');
    const texto = document.getElementById('texto');
    const lista = document.getElementById('lista');

    form.addEventListener('submit', e => {
      e.preventDefault();
      const t = texto.value.trim();
      if (!t) return;
      const li = document.createElement('li');
      li.textContent = t;
      lista.append(li);
      texto.value = '';
      li.dispatchEvent(new CustomEvent('tareas:agregada', {
        bubbles: true,            // sube hasta document
        detail: { texto: t }
      }));
    });

    document.addEventListener('tareas:agregada', e => {
      log('tareas:agregada → ' + e.detail.texto);
    });
  </script>
</body>
</html>
```

Ejercicio 15: ej15_contador.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 15 - Módulos desacoplados</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <p>Pendientes: <strong id="contador">0</strong></p>
  <ul id="lista"></ul>
  <script>
    const form = document.getElementById('form');
    const texto = document.getElementById('texto');
    const lista = document.getElementById('lista');

    function emitir(nombre, detail) {
      lista.dispatchEvent(new CustomEvent(nombre, {
        bubbles: true, detail
      }));
    }

    form.addEventListener('submit', e => {
      e.preventDefault();
      const t = texto.value.trim();
      if (!t) return;
      const li = document.createElement('li');
      li.textContent = t + ' ';
      const b = document.createElement('button');
      b.textContent = '×';
      b.onclick = () => {
        li.remove();
        emitir('tareas:eliminada', { texto: t });
      };
      li.append(b);
      lista.append(li);
      texto.value = '';
      emitir('tareas:agregada', { texto: t });
    });

    // ---- Módulo contador: no conoce a la lista ----
    const total = document.getElementById('contador');
    document.addEventListener('tareas:agregada', () => {
      total.textContent = Number(total.textContent) + 1;
    });
    document.addEventListener('tareas:eliminada', () => {
      total.textContent = Number(total.textContent) - 1;
    });
  </script>
</body>
</html>
```

Ejercicio 16: ej16_cancelable.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 16 - Evento cancelable</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <p id="motivo" role="alert" hidden></p>
  <ul id="lista"></ul>
  <script>
    const form = document.getElementById('form');
```

```
    const texto = document.getElementById('texto');
    const lista = document.getElementById('lista');

    // ---- Módulo lista ----
    form.addEventListener('submit', e => {
      e.preventDefault();
      const t = texto.value.trim();
      const antes = new CustomEvent('tareas:antes-de-agregar', {
        cancelable: true,
        detail: { texto: t }
      });
      // dispatchEvent devuelve false si alguien canceló
      if (!document.dispatchEvent(antes)) return;
      const li = document.createElement('li');
      li.textContent = t;
      lista.append(li);
      texto.value = '';
    });

    // ---- Módulo validador (independiente) ----
    const motivo = document.getElementById('motivo');
    document.addEventListener('tareas:antes-de-agregar', e => {
      const ok = e.detail.texto.length >= 3;
      motivo.hidden = ok;
      if (!ok) {
        e.preventDefault();
        motivo.textContent =
          'Rechazada: mínimo 3 caracteres (escribió ' +
          e.detail.texto.length + ').';
      }
    });
  </script>
</body>
</html>
```

Ejercicio 17: ej17_retiro.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 17 - Retiro de escuchadores</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <p>Pendientes: <strong id="contador">0</strong></p>
  <button id="desactivar">Desactivar contador</button>
  <ul id="lista"></ul>
  <pre id="consola"></pre>
  <script>
    const consola = document.getElementById('consola');
    const log = m => {
      console.log(m);
      consola.textContent += m + '\n';
    };
    const form = document.getElementById('form');
    const texto = document.getElementById('texto');
    const lista = document.getElementById('lista');
    function emitir(nombre, detail) {
      lista.dispatchEvent(new CustomEvent(nombre, {
        bubbles: true, detail
      }));
    }
    form.addEventListener('submit', e => {
      e.preventDefault();
      const t = texto.value.trim();
      if (!t) return;
      const li = document.createElement('li');
      li.textContent = t;
      lista.append(li);
      texto.value = '';
      emitir('tareas:agregada', { texto: t });
```

```
    });

    // ---- Contador con AbortController ----
    const total = document.getElementById('contador');
    const ctl = new AbortController();
    const { signal } = ctl;
    document.addEventListener('tareas:agregada', () => {
      total.textContent = Number(total.textContent) + 1;
    }, { signal });

    // Escuchador de una sola vez
    document.addEventListener('tareas:agregada', () => {
      log('once: responde solo a la 1.ª tarea');
    }, { once: true });

    // Escuchador permanente de control
    document.addEventListener('tareas:agregada', e => {
      log('agregada: ' + e.detail.texto);
    });

    desactivar.onclick = () => {
      ctl.abort();   // retira todo lo registrado con signal
      log('Contador desactivado (abort)');
    };
  </script>
</body>
</html>
```

Ejercicio 18: ej18_gestor.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 18 - Servicio con EventTarget</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Lista de tareas</h1>
  <form id="form">
    <input id="texto" placeholder="Nueva tarea">
    <button>Agregar</button>
  </form>
  <ul id="lista"></ul>
  <script>
    class GestorTareas extends EventTarget {
      #tareas = [];
      #sig = 0;
      agregar(texto) {
        const t = { id: ++this.#sig, texto };
        this.#tareas.push(t);
        this.dispatchEvent(
          new CustomEvent('tareas:agregada', { detail: t }));
        return t;
      }
      eliminar(id) {
        const i = this.#tareas.findIndex(t => t.id === id);
        if (i < 0) return;
        const [t] = this.#tareas.splice(i, 1);
        this.dispatchEvent(
          new CustomEvent('tareas:eliminada', { detail: t }));
      }
    }

    // ---- Interfaz: solo escucha y dibuja ----
    const gestor = new GestorTareas();
    const lista = document.getElementById('lista');
    const texto = document.getElementById('texto');

    gestor.addEventListener('tareas:agregada', e => {
      const li = document.createElement('li');
      li.dataset.id = e.detail.id;
      li.textContent = e.detail.texto + ' ';
      const b = document.createElement('button');
      b.textContent = '×';
      b.onclick = () => gestor.eliminar(e.detail.id);
      li.append(b);
```

```
      lista.append(li);
    });
    gestor.addEventListener('tareas:eliminada', e => {
      lista.querySelector(`[data-id="${e.detail.id}"]`)
        ?.remove();
    });
    document.getElementById('form').onsubmit = e => {
      e.preventDefault();
      if (texto.value.trim()) gestor.agregar(texto.value.trim());
      texto.value = '';
    };
  </script>
</body>
</html>
```

Ejercicio 19: ej19_vencida.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 19 - Temporizador y evento combinados</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Tareas con tiempo límite</h1>
  <form id="form">
    <input id="texto" placeholder="Tarea">
    <input id="limite" type="number" min="1" value="3"
      style="max-width:70px" title="segundos">
    <button>Agregar</button>
  </form>
  <p id="aviso" role="alert" hidden></p>
  <ul id="lista"></ul>
  <script>
    const lista = document.getElementById('lista');
    const texto = document.getElementById('texto');
    const limite = document.getElementById('limite');
    const temporizadores = new Map();   // id → id del setTimeout
    let sig = 0;

    function agregar(t, seg) {
      const id = ++sig;
      const li = document.createElement('li');
      li.dataset.id = id;
      li.textContent = `${t} (${seg} s) `;
      const b = document.createElement('button');
      b.textContent = '×';
      b.onclick = () => eliminar(id, li);
      li.append(b);
      lista.append(li);
      temporizadores.set(id, setTimeout(() => {
        temporizadores.delete(id);
        li.dispatchEvent(new CustomEvent('tareas:vencida', {
          bubbles: true,
          detail: { id, texto: t, limite: seg }
        }));
      }, seg * 1000));
    }
    function eliminar(id, li) {
      clearTimeout(temporizadores.get(id));   // evita avisos fantasma
      temporizadores.delete(id);
      li.remove();
    }
    document.getElementById('form').onsubmit = e => {
      e.preventDefault();
      const t = texto.value.trim();
      if (t) agregar(t, Number(limite.value) || 1);
      texto.value = '';
    };

    // ---- Componente de avisos (solo escucha) ----
    const aviso = document.getElementById('aviso');
    document.addEventListener('tareas:vencida', e => {
      aviso.textContent =
        `Venció «${e.detail.texto}» (${e.detail.limite} s)`;
      aviso.hidden = false;
```

```
    });
  </script>
</body>
</html>
```

Ejercicio 20: ej20_etl.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ej. 20 - Monitor ETL simulado</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Monitor ETL</h1>
  <div class="fila">
    <select id="origen">
      <option value="">— elija un origen —</option>
      <option>ventas.csv</option>
      <option>clientes.json</option>
    </select>
    <button id="iniciar">Iniciar</button>
    <button id="cancelar">Cancelar</button>
  </div>
  <progress id="barra" max="100" value="0"></progress>
  <pre id="bitacora"></pre>
  <script>
    class MonitorETL extends EventTarget {
      #id = null;
      #etapas = ['Extracción', 'Transformación', 'Carga'];
      #total = 6000;   // duración simulada (ms)
      #emitir(n, detail = {}) {
        this.dispatchEvent(new CustomEvent('etl:' + n, { detail }));
      }
      iniciar(origen) {
        if (this.#id !== null) return false;
        const antes = new CustomEvent('etl:antes-de-iniciar', {
          cancelable: true, detail: { origen }
        });
        if (!this.dispatchEvent(antes)) {
          this.#emitir('rechazado', { motivo: 'sin origen' });
          return false;
        }
        const t0 = performance.now();
        let previa = -1;
        this.#emitir('inicio', { origen });
        this.#id = setInterval(() => {
          const p = Math.min(1, (performance.now() - t0) / this.#total);
          const e = Math.min(2, Math.floor(p * 3));
          if (e !== previa) {
            previa = e;
            this.#emitir('etapa', { nombre: this.#etapas[e] });
          }
          this.#emitir('progreso', { pct: Math.round(p * 100) });
          if (p >= 1) {
            clearInterval(this.#id);
            this.#id = null;
            this.#emitir('fin');
          }
        }, 200);
        return true;
      }
      cancelar() {
        if (this.#id === null) return;
        clearInterval(this.#id);
        this.#id = null;
        this.#emitir('cancelado');
      }
    }

    // ---- Interfaz ----
    const etl = new MonitorETL();
    const barra = document.getElementById('barra');
    const bitacora = document.getElementById('bitacora');
    const anotar = m => bitacora.textContent += m + '\n';
```

```
    // Validador independiente: exige un origen
    etl.addEventListener('etl:antes-de-iniciar', e => {
      if (!e.detail.origen) e.preventDefault();
    });
    etl.addEventListener('etl:rechazado', () =>
      anotar('✖ Inicio rechazado: elija un origen'));
    etl.addEventListener('etl:inicio', e =>
      anotar('▶ Inicio con ' + e.detail.origen));
    etl.addEventListener('etl:etapa', e =>
      anotar('● Etapa: ' + e.detail.nombre));
    etl.addEventListener('etl:progreso', e =>
      barra.value = e.detail.pct);
    etl.addEventListener('etl:fin', () => anotar('✔ Proceso terminado'));
    etl.addEventListener('etl:cancelado', () => anotar('■ Cancelado'));

    iniciar.onclick = () => etl.iniciar(origen.value);
    cancelar.onclick = () => etl.cancelar();
  </script>
</body>
</html>
```

estilos.css

```
body{font-family:system-ui,sans-serif;max-width:480px;margin:16px auto;padding:0 12px;color:#1f2937}
h1{font-size:20px}
form,.fila{display:flex;gap:6px;margin:8px 0}
input,select{flex:1;padding:6px}
button{padding:6px 10px;cursor:pointer}
ul{padding-left:20px}
li{margin:4px 0}
[role=status],[role=alert]{background:#e6f4ea;border:1px solid #34a853;padding:6px 10px;border-
radius:4px;margin:6px 0}
[role=alert]{background:#fdecea;border-color:#d93025}
#aviso[data-estado]::after{content:" · en pausa"}
[hidden]{display:none!important}
pre{background:#111827;color:#e5e7eb;padding:8px;border-radius:4px;font-size:12px;white-space:pre-
wrap;min-height:40px}
.visor{font-size:44px;font-family:monospace;text-align:center;margin:12px 0}
progress{width:100%;height:18px}
```